
Kubernetes Lab: Imperative Commands (Pods, Deployments & Services)

Lab Objective

By the end of this lab, you will be able to:

- Create Pods using imperative kubectl commands
- Create a Deployment with replicas
- Expose a Deployment using NodePort and LoadBalancer services
- Verify and test Kubernetes resources

Prerequisites

- Kubernetes cluster (Minikube / Kind / EKS / AKS / GKE)
- kubectl configured and working
- Basic Linux command knowledge

Lab Environment Check

Verify cluster access:

kubectl cluster-info

Check nodes:

kubectl get nodes

Task 1: Run a Pod (Imperative Command)

Step 1: Create a Pod

Create a single NGINX Pod:

kubectl run nginx-pod --image=nginx --restart=Never

Step 2: Verify the Pod

kubectl get pods

Describe the Pod:

kubectl describe pod nginx-pod

Access Pod logs:

kubectl logs nginx-pod

Task 2: Run a Pod with Replicas (Using Deployment)

> **Note:** Kubernetes does not support replicas for standalone Pods. Replicas are managed using **Deployments** or **ReplicaSets**.

Step 1: Create a Deployment with Replicas

kubectl run nginx-deploy --image=nginx --replicas=3

Step 2: Verify Deployment and Pods

kubectl get deployments

kubectl get pods

Check replica details:

kubectl describe deployment nginx-deploy

Task 3: Create a Deployment (Explicit)

Create a Deployment with a specific container port:

kubectl create deployment web-deploy --image=nginx

Expose container port:

kubectl set image deployment/web-deploy nginx=nginx

Verify:

kubectl get deployment web-deploy

Scale the Deployment:

kubectl scale deployment web-deploy --replicas=4

Confirm scaling:

kubectl get pods

Task 4: Expose Deployment Using NodePort Service

Step 1: Expose Deployment

```
kubectl expose deployment web-deploy \
  --type=NodePort \
  --port=80
```

Step 2: Verify Service

kubectl get services

Describe service:

kubectl describe service web-deploy

Step 3: Access Application

Get Node IP and NodePort:

kubectl get nodes -o wide

Access in browser or curl:

curl http://<NODE-IP>:<NODE-PORT>

Task 5: Expose Deployment Using LoadBalancer Service

Step 1: Create LoadBalancer Service

```
kubectl expose deployment web-deploy \
  --type=LoadBalancer \
  --port=80 \
  --name=web-lb
```

Step 2: Verify LoadBalancer

kubectl get services

Wait for **EXTERNAL-IP** to be assigned.

> ⚠️ On Minikube, use:

minikube service web-lb

Step 3: Test Access

curl http://<EXTERNAL-IP>

Task 6: Cleanup Resources

Delete Services:

kubectl delete service web-deploy web-lb

Delete Deployments:

kubectl delete deployment nginx-deploy web-deploy

Delete Pod:

kubectl delete pod nginx-pod

Verify cleanup:

kubectl get all

Key Takeaways

- **Imperative commands** are fast and ideal for labs and troubleshooting
- **Pods** run single containers
- **Deployments** manage replicas and scaling
- **NodePort** exposes services via node IP
- **LoadBalancer** exposes services externally (cloud environments)
